Q7 – Concurrent, Fault-Tolerant Result Processor

Stage 1 – Interpret (Joint)

- **Parallel tasks:** each file (read → validate → compute result) is independent.
- **Shared data:** the results collection and the success/failure counters, plus the summary.
- **Failures and handling:** | Failure | Exception | Handling | |---|---|---| | Missing file | `FileNotFoundException` | Record as FAILED(missing), continue | | Corrupt format | `NumberFormatException` / custom `InvalidScoreException` | Record FAILED(corrupt), continue | | Out-of-range marks | custom `ValidationException` | Record FAILED(invalid) | | I/O error | `IOException` | Record FAILED(io), optionally retry | | Unexpected bug | `RuntimeException` | Caught in task, recorded | | Task cancelled/timeout | `InterruptedException`/`TimeoutException` | Record, restore interrupt flag |
- **Measurable requirements:** (R1) process 10,000 files; (R2) one bad file must not abort the run (0 aborted runs); (R3) success + failure count = 10,000 always; (R4) summary produced 100% of runs; (R5) total time with the chosen pool ≤ X s `<fill>` and at least Y× faster than 1 thread.

Stage 2 – Design (Student A)

- **Class diagram:**

```
ResultProcessor ──uses──> ExecutorService
ResultProcessor ──creates──> FileTask implements
Callable<Outcome>
Outcome { fileName, status(OK|FAILED), marks/result, errorMessage
}
Summary { successCount, failureCount, failures list }
```

- **Pseudocode:**

```
pool = fixed(p)
futures = []
for f in files: futures.add(pool.submit(new FileTask(f)))
for fut in futures:
    try   outcome = fut.get()          // never throws for bad
file; task returns Outcome
    catch ExecutionException e   -> record FAILED(e.getCause())
    catch InterruptedException   -> restore flag, break
pool.shutdown(); awaitTermination
print summary
FileTask.call(): try { read; validate; compute; return OK } catch
(Exception e) { return FAILED(e) }
```

- **Concurrent collection:** `ConcurrentHashMap<String,Outcome>` (keyed by file; thread-safe, no global lock, no duplicates), with `AtomicInteger`/`LongAdder` for counters. Alternative: `ConcurrentLinkedQueue` or collecting results from the `Futures` in the main thread, which needs no shared mutable state at all.

- **One failure doesn't stop the run:** exceptions are caught inside `call()` (and `ExecutionException` is handled per future in the collector loop), so every future yields an outcome.

Stage 3 – Implementation (Student B)

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;

class Outcome {
    final String file; final boolean ok; final double result; final
String error;
    Outcome(String f, boolean ok, double r, String e) { file = f;
this.ok = ok; result = r; error = e; }
}

class FileTask implements Callable<Outcome> {
    private final Path path;
    FileTask(Path p) { path = p; }
    public Outcome call() {
        try {
            List<String> lines = Files.readAllLines(path);          //
NoSuchFileException / IOException
            if (lines.isEmpty()) throw new
IllegalArgumentException("empty file");
            double sum = 0;
            for (String l : lines) {
                double m = Double.parseDouble(l.trim());
// NumberFormatException
                if (m < 0 || m > 100) throw new
IllegalArgumentException("marks out of range: " + m);
                sum += m;
            }
            return new Outcome(path.toString(), true, sum /
lines.size(), null);
        } catch (Exception e) {
            return new Outcome(path.toString(), false, 0,
e.getClass().getSimpleName() + ": " + e.getMessage());
        }
    }
}

public class ResultProcessor {
    static long run(List<Path> files, int threads, boolean print)
throws Exception {
        long start = System.nanoTime();
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        List<Future<Outcome>> futures = new ArrayList<>();
        for (Path p : files) futures.add(pool.submit(new
```

```
FileTask(p)));
        ConcurrentHashMap<String, Outcome> results = new
ConcurrentHashMap<>();
        int ok = 0, bad = 0;
        for (Future<Outcome> f : futures) {
            try {
                Outcome o = f.get();
                results.put(o.file, o);
                if (o.ok) ok++; else bad++;
            } catch (ExecutionException e) {
                bad++;                                                  //
unexpected task failure
            }
        }
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
        long ms = (System.nanoTime() - start) / 1_000_000;
        if (print) System.out.println("Success=" + ok + " Failed=" +
bad + " Total=" + (ok + bad));
        return ms;
    }

    public static void main(String[] args) throws Exception {
        Path dir = Files.createTempDirectory("scores");
        List<Path> files = new ArrayList<>();
        for (int i = 0; i < 10000; i++) {
            Path p = dir.resolve("s" + i + ".txt");
            if (i % 50 == 0) { /* missing: don't create */ }
            else if (i % 40 == 0) Files.writeString(p, "abc\n");
// corrupt
            else Files.writeString(p, "70\n80\n90\n");
            files.add(p);
        }
        for (int t : new int[]{1, 2, 4, 8}) {
            long ms = run(files, t, t == 1);
            System.out.println("threads=" + t + " time=" + ms + "
ms");
        }
    }
}
```

- **Hand-check:** make a set of 10 files (e.g. 6 good, 2 corrupt, 2 missing) and confirm `Success=6 Failed=4 Total=10`.
- **Measurements:** record the table of pool size vs. time `<fill>` (repeat each 5 times and average, discarding the first run for JIT warm-up). Record anything not handled (e.g. a file with a BOM, an empty file or a directory in place of a file) and fix it.

Stage 4 – Defend (Joint)

- **Recommended pool size:** for I/O-bound work about 2× cores up to a measured knee (typically 4–8 on a laptop); for CPU-bound work ≈ number of cores. Choose from your measured table, the point where more threads stop reducing time. More threads don't always help because of limited cores, disk bandwidth saturation, context switching, memory for thread stacks and lock contention on shared collections.
- **Complete and correct summary:** every submitted task returns exactly one `Outcome` (exceptions are converted into FAILED outcomes), the collector waits on **every** `Future` with `get()`, counters are updated only by the collecting thread (or atomic), and `shutdown` + `awaitTermination` ensures nothing is left running, so `ok + bad == number of files`.
- **Limitation (example):** if the JVM crashes or the process is killed there is no checkpointing, so everything restarts. Also all outcomes sit in memory, and a hung read of a network file could block `get()` forever (needs `get(timeout)`).

Alternative: parallel streams / thread per file

```
// parallel stream
files.parallelStream().map(p -> new
FileTask(p).call()).collect(Collectors.toList());
// thread per file
for (Path p : files) new Thread(() -> ...).start();
```

- **Parallel streams:** use the common ForkJoinPool (size ≈ cores−1), so you can't easily tune the pool size, and blocking I/O inside it starves other parallel stream users. Time ≈ N/p for p = common-pool parallelism. Memory ≈ p × stack (small). A failure not caught inside the lambda propagates and can abort the whole terminal operation, so you must catch inside the mapper (as `FileTask.call()` does).
- **Thread per file:** creates up to 10,000 threads; memory ≈ 10,000 × stack size (≈1 MB reserved each, so ~10 GB of virtual memory) with a risk of `OutOfMemoryError: unable to create native thread`; heavy context switching; time often worse than a pool. Failure handling is per thread, so an uncaught exception just kills that thread (results lost unless caught).
- **ExecutorService:** time ≈ N/p, memory ≈ p × stack, with per-task `Future` exception capture and tunable pool size. This is the best fit.